

add_rms_norm_bias 融合算子 Ascend 性能优化报告

1. 背景

RMSNorm 是 Transformer 每层 decoder 的必经算子；带残差连接的推理引擎里，归一化前还要先把 attention/FFN 的输出与 residual 相加。把「残差相加 → RMSNorm → bias」融为一个 kernel，可以把中间 x 的往返读写省掉，并将三次 launch 合为一次。vLLM Ascend 插件据此实现了自研算子 `add_rms_norm_bias`：一次调用产出三份输出（归一化结果 y、统计量 rstd、相加后的残差 x），由 `AscendRMSNorm.forward_out` 在 decoder 层调用，覆盖 prefill 与 decode 场景。

本文总结 `add_rms_norm_bias` 在 Ascend 910B3 上的性能优化过程，重点分析 bf16 NORMAL 生产主路径的发射线程停等、冗余向量遍历与行间串行问题。优化包括 rstd 全程向量计算、 γ/β 类型转换上提及双缓冲行间流水，并扩展到 fp16 与 decode 路径。报告给出基线画像、逐轮性能结果、正确性修复及最终适用范围，同时记录 reduce 依赖链重构未取得显著收益的实验，区分设备观测与待验证的机理解释。

测试环境

项目	值
设备	Ascend 910B3 (dav_c220 向量核)，40 AIV，UB 192KiB/核
CANN	9.1.0
模型场景	DeepSeek 类 hidden=7168，bf16 主 / fp16 对照
测试矩阵	tokens = 1/4/16/64/128/512/1024/2048/4096 × bf16/fp16
计时口径	NPUGraph capture+replay (≈背靠背 serving，5 样本取中位)；msprof op 做管线归因

算子功能

```
x      = bf16(fp32(x1) + fp32(x2))           # 残差相加 (c220 无 bf16 元级加法，须在 fp32 域)
rstd   = 1/sqrt(mean(x²) + eps)              # fp32
y      = bf16( fp32(bf16(x_norm)) × fp32(γ) + fp32(β) ) # 双舍入契约
返回 (y, rstd, x) 三份输出
```

结论摘要

项目	结论
生产主路径	bf16、hidden=7168、rows≥41 → tiling key 30 (NORMAL 变体)；decode (rows≤40) → SINGLE_N
基线瓶颈	发射线程停等 (scalar_wait 占墙钟 85%) + 行间零流水 + 15 遍全宽 pass 中 3 遍冗余
核心方案	消 V_S/S_V 往返 (rstd 全程 V 域)、双缓冲行间流水、 γ/β cast 上提每核
最终收益	bf16 @2048 -24.8% (119.4→89.8μs)、@4096 -26.8%；fp16 @2048 -4.1%
最终状态	有效带宽约 1.31TB/s (名义峰值 ~1.6TB/s 的 82%)，接近带宽/向量流水转折区
正确性	精度测试通过 (原 126 用例 + 新增 154 用例)
已回退方案	Reduce 多链交错重构：收益噪声级且存在 dst==src1 交错 fault 风险 (见 §4.6)

2. 算子在大模型中的位置与数据

2.1 位置

```
token embeddings
→ N × decoder layer (每层两次归一化，带残差时都走本算子)
    attention/FFN 输出 x1 + residual x2 → RMSNorm → bias → y
→ logits

调用方: AscendRMSNorm.forward_out; prefill (大 tokens) 与 decode (1~4 tokens) 都命中
```

2.2 变体分发——本算子的第一课：先确认优化对象

算子按 (dtype_key×10 + mode_key) 分发 5 个变体 (NORMAL/SPLIT_D/MERGE_N/SINGLE_N/MULTI_N)。任务简报认定「hidden=7168 走 SPLIT_D」，但静态读 tiling 推出的是另一回事，最终以设备日志实证 (ASCEND_GLOBAL_LOG_LEVEL=1 采集 tiling 输出)：

基线生产 shape (hidden=7168)	tiling key	变体	说明
bf16, rows ≤ 40	33	SINGLE_N	decode 档
bf16, rows ≥ 41	30	NORMAL	生产 prefill 主路径
fp16, rows ≤ 40	13	SINGLE_N	
fp16, rows ≥ 41	14	MULTI_N	fp16 prefill, 基线已采用双缓冲

基线 SPLIT_D 的触发条件是 numCol > ubFactor (有 β 时 bf16 上限 11264 列)，生产 hidden=7168 未命中该路径。最终版本还对 NORMAL 候选路径增加 UB 成本检查，超限时回退 SPLIT_D (见 §4.5)；7168 仍走上述生产路径。**如果按简报直接优化 SPLIT_D，生产 shape 上的工作量会落在零收益的路径上。**教训：分发条件要用实测 tiling 日志钉死，不要信文档/口口相传。

2.3 输入输出与片上数据流

张量	形状	类型	说明
x1 / x2	[tokens, 7168]	bf16	输入与残差
y / β	[7168]	bf16	每核只载一次
y / x	[tokens, 7168]	bf16	输出
rstd	[tokens, 1]	fp32	统计量输出

原始实现 (NORMAL 变体) **每行全串行**：单缓冲队列使下一行的 MTE2 装载必须等上一行的 V 消费完；每行末尾一次 V_S→GetValue→S_V 标量往返把发射线程钉在行尾。bf16 的 y 链在 fp32 域做数学 (c220 无 bf16 元级乘加)，每行 **15 遍全宽 pass**——其中 3 遍冗余 (y/β 的 cast 每行重做、1/N 在 reduce 前全宽乘)。

优化后的片上数据流（R2 流水化后）：

```
每行 i (槽位 s = i&1) :
MTE2: x1[i+1]/x2[i+1] 预取进另一槽    ← 与本行 V 链重叠
V:    cast x1/x2 → add → rint = x_out (落 x1[s])
      mul 平方 → reduce → 标量链(1 elem) → rstd 留在 V 域
      Gather 广播 rstd → mul → rint = y-mid → widen → xγ → +β → rint = y (落 x2[s])
MTE3: x_out 与 y 落盘                ← 与下一行 V 链重叠
rstd: 1 elem Adds 进 8-lane 累积块 → 块末 Gather 压缩 → 一次 DataCopyPad 落 GM
```

消除每行 `v_s/s_v` 往返，将依赖等待放到消费 pipe；发射线程的 `scalar_wait` 显著降低，并非所有等待都归零。

3. 优化总览

轮次	手段	针对的问题	分析依据
第一轮-减法	消 V_S/S_V 标量往返； γ/β cast 上提每核；1/N 折叠进标量；tiling ubFactor 11264→7168	15 遍全宽 pass 中 3 遍冗余 + 发射线程停等	代码走读 + msprof
第二轮-流水	$x[2]/x2[2]$ 双缓冲（局部行号奇偶）、事件四件套三方向单在途、rstd 全程 V 域	行间零流水（单缓冲串行）、 <code>scalar_wait</code> 85%	缓冲与事件依赖分析 + msprof
第三轮-广度	NORMAL 序言重叠；MULTI_N 折叠 1/N；SINGLE_N 消往返	64-128 档序言占比、fp16/decode 路径	各变体代码走读 + 性能对照
检视修复	外部检视 7 条全修（UB 成本模型、rstd 压缩、溢出域、host 校验）+ 新增精度测试	正确性（原有测试未检出的行混叠/越界）	外部检视 + 设备复现
负结果	Reduce 依赖链重构（8 链交错）	假设的 reduce 延迟链	当前配置下收益为噪声级

经验法则：先实证 tiling 命中的变体（优化错变体 = 零收益）→ 核对数值契约，再消标量往返与冗余 pass → 行间流水（本例收益最大，需明确槽位复用与事件生命周期）→ 用未动路径与独立复测估计噪声 → 结合 HBM 理想下限与向量占用校准后续预期。

4. 逐轮优化过程

4.1 初始版本：每行全串行，双瓶颈叠加

瓶颈判定（msprof op, 2048×7168 bf16, 40 核均值，墙钟 124.4μs）：

指标	值	占墙钟	判读
vec busy	87.9μs	70.7%	最大占用者但未饱和（1.69μs/行 ÷ 15 pass ≈ 0.11μs/pass）
mte2 / mte3 busy	27.0 / 18.7μs	21.7% / 15.0%	未与 V 充分重叠
scalar_wait	106.0μs	85.2%	发射线程几乎全程停等

两个信号拼出双瓶颈结构：V 是最大占用者，但其 busy 时间与任务墙钟仍有约 29.3% 的差距；`scalar_wait` 85.2% 说明发射线程被每行的标量往返与单缓冲队列的串行点钉住——它停着，装载/落盘的重叠就难以展开。CSV 中 `aiv_vec_ratio` 的 40 核均值为 73.1%，分母是各核 AIV 活跃时间，不是任务墙钟，不能与表中占比混用。注意口径：**各 pipe 的 busy/wait 统计**

在时间上可能重叠，不是互斥占比，不能相加得到 100%。

代码走读量化清单 (bf16 每行，按类别计数)：

类别	基线	第一轮后	变化	明细
全宽向量 pass (≥64 lane 宽度)	15	12	-3	cast x1、cast x2、add、rint=x 输出、mul 平方、 mul_s × 1/N (折叠) 、reduce 树、mul _s × rstd、rint y-mid、重新 widen、 cast y (上提) 、mul γ、 cast β (上提) 、add β、rint y 输出
单元素向量/标量操作	~6	~6	不变	Adds(eps)/Sqrt/Duplicate(ONE)/Div + rstd 累积
V/S 跨流水同步	2 次/行	0	-2 次/行	V_S→GetValue→S_V→SetValue 整链消除 (rstd 留在 V 域)
GM 读写量/行	4 份全宽张量 + rstd	4 份全宽张量 + rstd	不变	读 x1/x2, 写 x/y; rstd 为每行 1 个 fp32 值、每 64 行批量落盘

(“15 遍全宽”只计全宽向量遍历，不含单元素操作与同步步骤；15→12 为第一轮状态，最终 bf16/fp32 已恢复 reduce 前的全宽 1/N 缩放，见 §4.5。)

4.2 第一轮：消标量往返与冗余 pass

问题分析

三处冗余 + 一处停等：γ/β 每核只载一次却每行 cast；1/N 在 reduce 前全宽乘；rstd 经 V_S/GetValue/S_V/SetValue 绕行标量域。每行 15 遍全宽 → 12 遍，且消掉每行 2 次跨 pipe 同步。

关键变化

```
// ④ rstd 广播留在向量域 (位级一致)：Gather 复制 sqx[0] 成 8-lane 块，
//   stride=0 块广播 Mul 替代标量操作数的 Muls—rstd 不再走 S pipe
__aicore__ inline void MulByRstd(dst, src, sqx, count) {
    Gather(rstd8, sqx, zeroOff8, 0, 8);
    PipeBarrier<PIPE_V>();
    Mul(dst, src, rstd8, 64, repeatTimes, {1, 1, 0, 8, 8, 0}); // src1RepStride=0
    ... // 尾块同参
}

// ⑤ γ/β 每核 widen 一次进常驻缓冲 (+57.4KB, 靠 tiling 收紧腾出)
// ⑥ 1/N 折叠: reduce 前全宽 Muls → reduce 后 1 元素 Muls
// ⑦ tiling: NORMAL 模式 ubFactor 11264 → numColAlign (原常量是 16B/列×11264
//   =180KB 的全宽安全设计; 收紧后缓冲随 numCol 比例, 为 cast 常驻腾 76KB)
```

结果

NPUGraph: @2048 119.4→109.3μs (-8.4%) , @512-4096 -7~8%; msprof wall 124.4→114.7μs、vec busy 87.9→77.8μs, mte2/mte3 不变——「纯减 vec 工作」的定位被数据确认。未动代码的 fp16 路径同轮波动 ±1μs → 该档噪声带，收益判定以 ≥512 档为准。

第一轮各档位提升率 (bf16, 基线 → 第一轮; 正为更快)

NPUGraph, hidden=7168; 提升率 = (基线 - 优化后)/基线

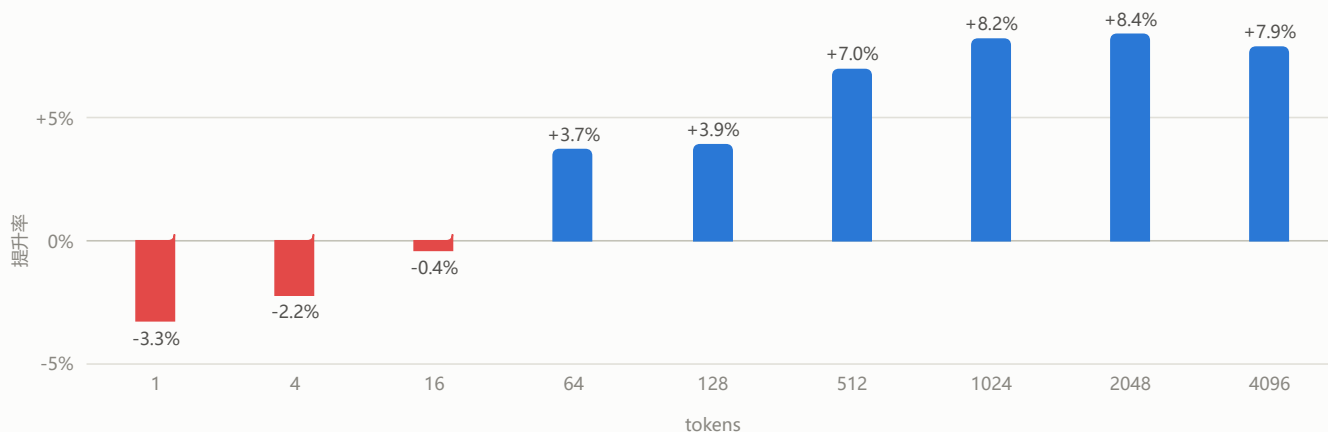


图 1 · 第一轮各档位提升率

4.3 第二轮：行间流水 (收益最大的一轮)

问题分析

R1 后 `aiv_vec_ratio` 约 70.0% (AIV 活跃时间口径; $77.8\mu\text{s} \div 114.7\mu\text{s}$ 的墙钟占比约 67.8%), `scalar_wait` 约占墙钟 84%: 单缓冲队列使行 $i+1$ 的装载等行 i 的 V 消费, 发射线程被逐行钉住。消除标量往返只是松绑, 行间流水才是把 vec 喂饱的手段。

关键变化

将 bf16 NORMAL 重写为行间双缓冲流水: 用局部行号奇偶选择槽位, 按

`AllocEventID/SetFlag/WaitFlag/ReleaseEventID` 管理事件生命周期, 并保证发射与消费条件一致、kernel 退出时无在途事件。数据流与同步关系如下:

槽位: $x1[2]/x2[2]$ (装载 + 兼作 x/y 输出), fp32 工作区单缓冲 (仅 V 触碰, V 有序天然安全); γ/β bf16 暂存借道 row-0 槽位, 一次 `V_MTE2` 守卫释放
事件: `MTE2_V` (装载→V)、`V_MTE2` (槽位释放→装载)、`MTE3_MTE2` (落盘→装载)
各单在途 + 每行一对 `V_MTE3`; wait 全落在消费 pipe (发射线程零阻塞);
发射条件与消费条件严格一致 ($i+2 < \text{rowWork}$), 退出零悬挂
rstd: 全程 V 域—reduce 后 1 elem Adds 进 8-lane/行累积块 (32B 对齐向量写),
每 rowFactor 行 Gather 压缩 + 单次 DataCopyPad 落 GM

结果

NPUGraph: @2048 109.3→**86.7 μs** (-20.7%), @4096 -21.9%, @512 -7.8%, ≤ 128 档持平 (每核行数少, 流水收益未超出噪声带)。msprof: wall **86.4 μs** (-30.5% vs 基线), `aiv_vec_ratio` 89.5% (AIV 活跃时间口径), `scalar_wait` 106→**1.86 μs** (墙钟占比约 85%→2%) ——发射线程停等显著减少, 三 pipe 忙碌和/墙钟 = 1.32。

指令级流水对比（另一次 msprof TimelineDetail 采集，单独统计，不与上文 PipeUtilization 读数合并）：

指标	流水前	流水后
wall	101.4μs	80.7μs
MTE2 忙碌	50.8%	94.2%
SCALAR 忙碌	90.0%	2.5%
三线并行度	2.27	2.87



精度测试：通过（当轮 126 用例）。

4.4 第三轮：广度轮——序言重叠与旁支变体

三处独立小改动，一轮闭环：① NORMAL 序言里 γ/β 的 MTE2 装载提前到 ι 预计算之前（64 次 Duplicate $\approx 1.6\mu s$ 发射与装载重叠），64-128 档受益；② MULTI_N 折叠 1/N（fp16+bf16 共用 ComputeRstd）；③ SINGLE_N 三 dtype 消 V_S/S_V （Gather 广播，复用已死的 reduce 工作区尾块）+ 折叠。

当轮结果：fp16 @512-2048 **-4.2~-5.3%**（MULTI_N）；bf16 decode 1-4 tokens 的记录读数为 **-2.6~-3.9%**（SINGLE_N），但位于小档噪声带内，不能认定为可靠收益；bf16 大档持平（序言已摊薄）。上述为第三轮历史读数，最终结果见 §4.7。

第三轮各档位提升率（相对第二轮；正为更快）

NPUGraph, hidden=7168; fp16@4096 为噪声带内的负读数

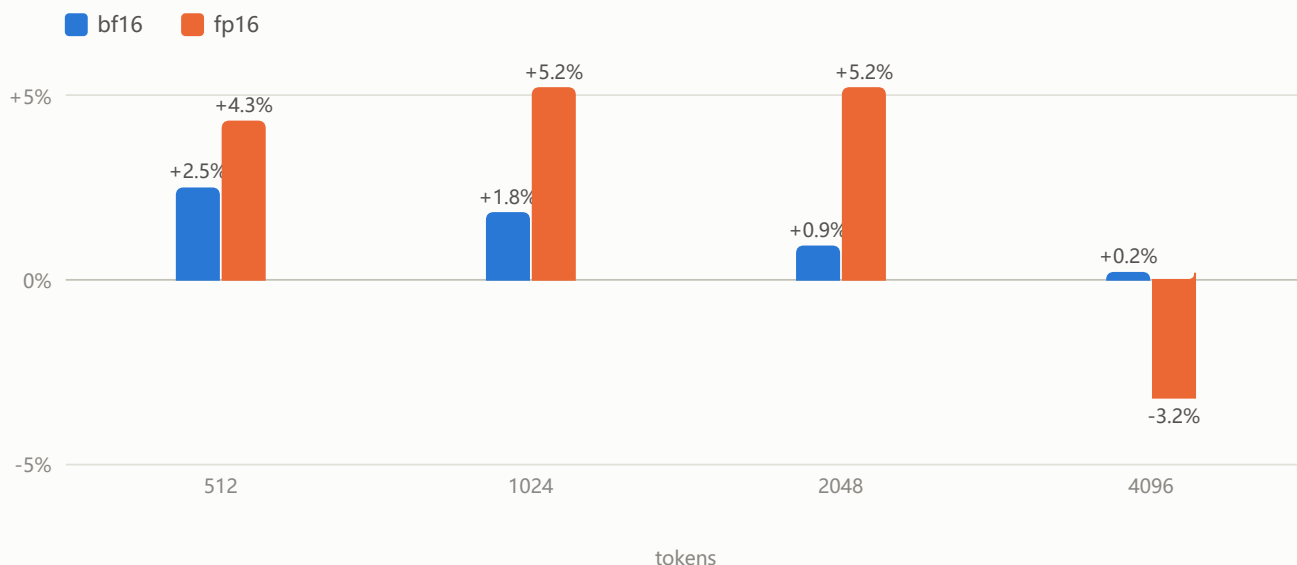


图 3 · 第三轮各档位提升率

4.5 检视修复轮：补齐正确性与边界验证

外部静态检视 7 条，全部采纳，其中三条是真缺陷且两条为本轮优化引入：

#	问题	处置与教训
1	rstd 压缩 Gather 索引按 $[b \times 32] \times 8$ 生成——偏移张量按字节逐元素消费， $n > 8$ 时行值重复 8 份	iota 改连续 $[0, 32, \dots]$ ，标量 SetValue 写入（逐元素 V-store 4B 非对齐当场 fault——修复引入又当场修复）；新增针对性回归
2	bf16 NORMAL 缓冲随 numCol 比例增长，hidden=8192 时预算 199,808B > 可用 195,584B（原 11264 常量实为 16B/列全宽安全设计）	tiling 按 dtype/ β 的每列字节成本模型驱动，超限回退 SPLIT_D；回退保持 colTileNum ≥ 2 （单 chunk 是 SPLIT_D 从未验证过的路径，设备实测每核一整行垃圾）
3	1/N 后移扩大溢出域：在 hidden=7168、 $x \approx 3e17$ 的回归输入下 raw sum(x^2) 为 Inf，而逐元素缩放的 golden 有限	fp32/bf16 回退折叠（@2048 +4.5% 为正确性代价）；fp16 保留（max raw sum $5.3e13 \ll$ fp32 max，有界证明入注释）
4-5,7	beta 无 shape/dtype 校验；epsilon 放过 NaN（NaN<0=false，且调用点丢弃返回值）；numel 无 32 位防护	host 侧补齐；epsilon 修复要点是让返回值真正被检查
6	原精度验证未充分覆盖数值与边界缺陷	新增独立精度测试（不动原文件），补行间对比、极值输入、UB 边界 col=8192、尾块 rows=313/2064/2600、无 β 、epsilon/beta 拒绝路径；测试通过

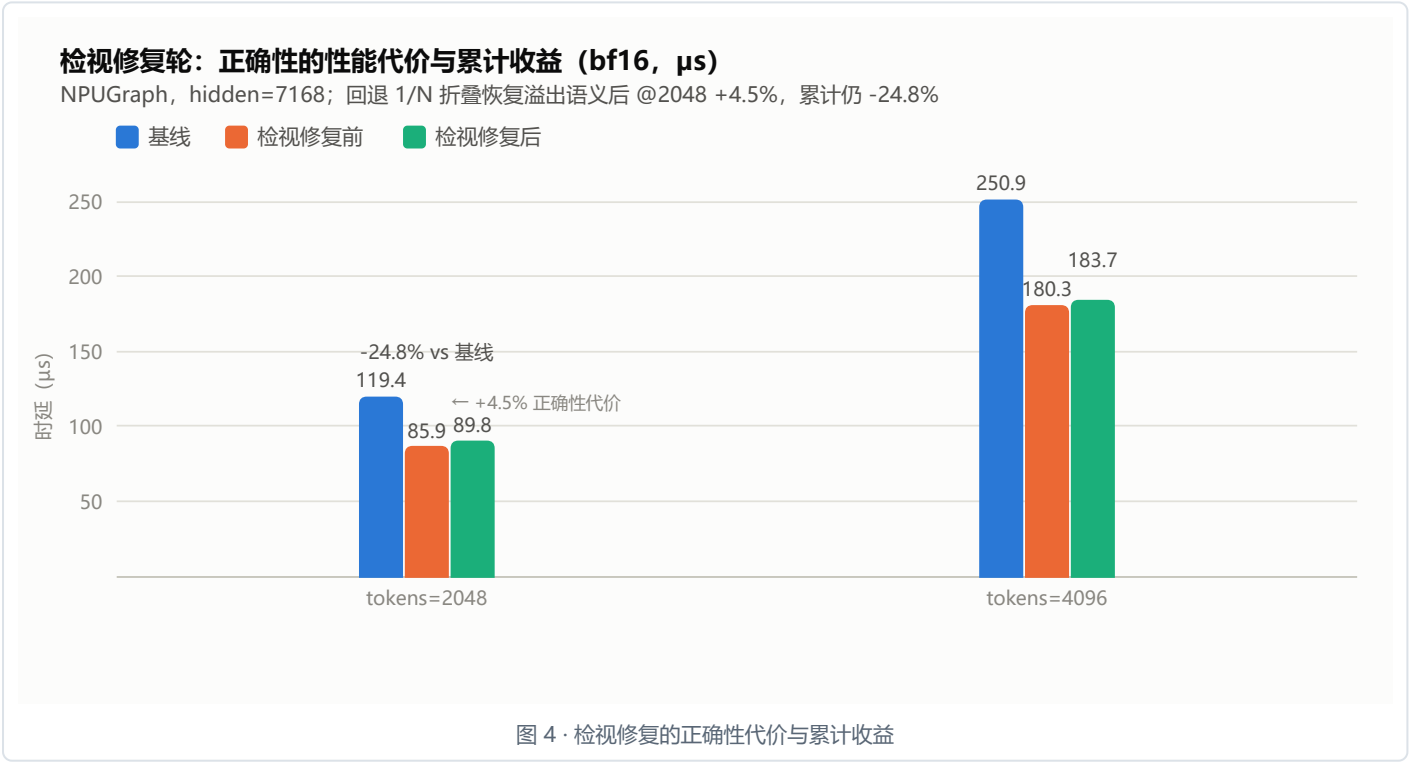
精度测试：通过（当轮原 126 用例 + 新增 150 用例；后续补充 4 用例，最终新增用例共 154 个）。代价：@2048 89.8 μ s（85.9 $\rightarrow$ +4.5%）。精度与边界验证先行，再评价性能收益。

优化后 **NORMAL** 路径 **UB 预算表** (bf16, hidden=7168; 单位统一 KiB=1024B, 可用 UB = 192KiB - 1KiB 系统保留 = 191KiB = 195,584B) :

缓冲	每列成本	hidden=7168 占用
x1[2] / x2[2] 双缓冲 (bf16, 各 2×7168×2B)	8 B	28 KiB ×2
xFp32Buf + sqxBuf (fp32 工作区)	8 B	28 KiB ×2
γ/β fp32 常驻	8 B	28 KiB ×2
rstdAcc (8-lane/行 × rowFactor=64)	—	2 KiB
iota + compBuf + reduceFp32	—	0.75 KiB
one / zeroOff / rstdBcast	—	0.09 KiB
合计	24 B/列	170.8 KiB (余量 20.2 KiB)

hidden=8192 推导: $24 \times 8192 + 3200 = 199,808B > 195,584B \rightarrow$ 超限。固定项 3200B 的构成: 实际缓冲 2912B (2KiB + 0.75KiB + 96B) + 288B 对齐/备用预留。回退阈值公式 (与代码 DetermineModeParameters 一致) :
 $numColAlign \leq (195584 - 3200) / 24 = 8016$ 列 ——对齐到 16 的倍数后 7168 走 NORMAL、8192 回退 SPLIT_D (7168 的编码常量余量 19.9 KiB) 。host 侧以同式实装为成本模型, kernel 不再自行假设预算。

最终精度测试结果: 通过 (原 126 用例 + 新增 154 用例; 按设备测试记录汇总) 。新增测试覆盖不同 dtype、行间对比、极值输入、压缩尾块、UB 边界、无 β 及非法参数路径。未覆盖项: 全零/Inf 输入、输入输出别名与原地约束。



4.6 负结果: Reduce 依赖链重构未取得显著收益

实验记录如下:

- 1. 假设: legacy 归约的单指令多 repeat Add(..., dstRepStride=0) 在同一累加块上形成 112 深 RAW 链, 顺序 V 管线应完整暴露该延迟 (估 0.3-0.6μs/行) ; 8 链交错发射可消除。

2. **实验变量**：新增 ReduceSumFP32Parallel (8×64-float 累加器、交错发射、显式两两折叠 + WRS)，仅切换流水化 bf16 路径的 reduce 调用，其余不动；过程中按二分法变换发射顺序与操作数位次 (V1/V2a/V2b，见下表)。
3. **控制变量**：数据、shape、设备、测量口径全部不变；每轮 clean build + md5 自检。
4. **设备观测**：

实验	发射顺序 / 操作数位次	观测
V1	交错发射，累加器在 src1 (dst==src1)	vector core exception (col=7168 无尾也崩 → 与尾路径无关)
V2a	链主体顺序发射 (其余同 V1)	通过 → 故障定位于交错本身
V2b	交错发射，累加器在 src0 (dst==src0)	通过 → 可用形式
V2b 最终版	精度测试通过 (当轮新增 150 用例 + 原 126 用例)	@2048 89.09 vs 89.81μs (-0.8%，噪声级)；vec busy 约 77.4μs，未观测到明显下降

5. **性能与正确性结果**：正确性零回退；性能收益 -0.8% 为噪声级——**当前配置下未观测到可测的依赖延迟收益**。观测与解释分开记录：设备观测是"交错 8 链与单指令 112-repeat 链等速"；对此的候选解释 (未做微架构级验证) 包括硬件对 repeat 间累加内部转发/按吞吐执行；静态分析发现的 0.44μs/行"未解释缺口"同样存在多个未验证候选 (cast 半吞吐、barrier 开销等)，不归于单一因素。
6. **结论** (按强度分级)：**性能实测**——交错 8 链重构在当前配置下收益为噪声级；**设备观测**——本轮 c220 配置中，交错发射的单 repeat Add 在 dst 轮转且 dst==src1 时触发 vector core exception，累加器改放 src0 位后通过 (顺序发射时 dst==src1 也通过)；**待验证假设**——多 repeat 累加的内部机制、依赖延迟是否在其他配置下可观测，以及 V 时间缺口的构成。
7. **回退及原因**：整体回退——收益噪声级、112 条标量发射/行 vs legacy 1 条 (issue 侧反而更重)、+1.75KB UB；负结果与机理留档，避免未来重复推导。

4.7 总收益与最终状态

累计优化 (NPUGraph, hidden=7168, μs)：

版本	@512	@1024	@2048	@4096
基线	33.2	59.0	119.4	250.9
+ 第一轮 (消往返/冗余 pass)	30.8	54.2	109.3	231.1
+ 第二轮 (行间流水)	28.4	47.9	86.7	180.6
+ 第三轮 (广度)	27.7	47.1	85.9	180.3
+ 检视修复 (溢出域回退)	29.4	49.5	89.8	183.7
累计	-11.4%	-16.2%	-24.8%	-26.8%

各版本时延演进 (bf16, hidden=7168, NPUGraph)

横轴 tokens ($\log_2$) ; 纵轴单次调用耗时 μs ; @2048 累计 -24.8%

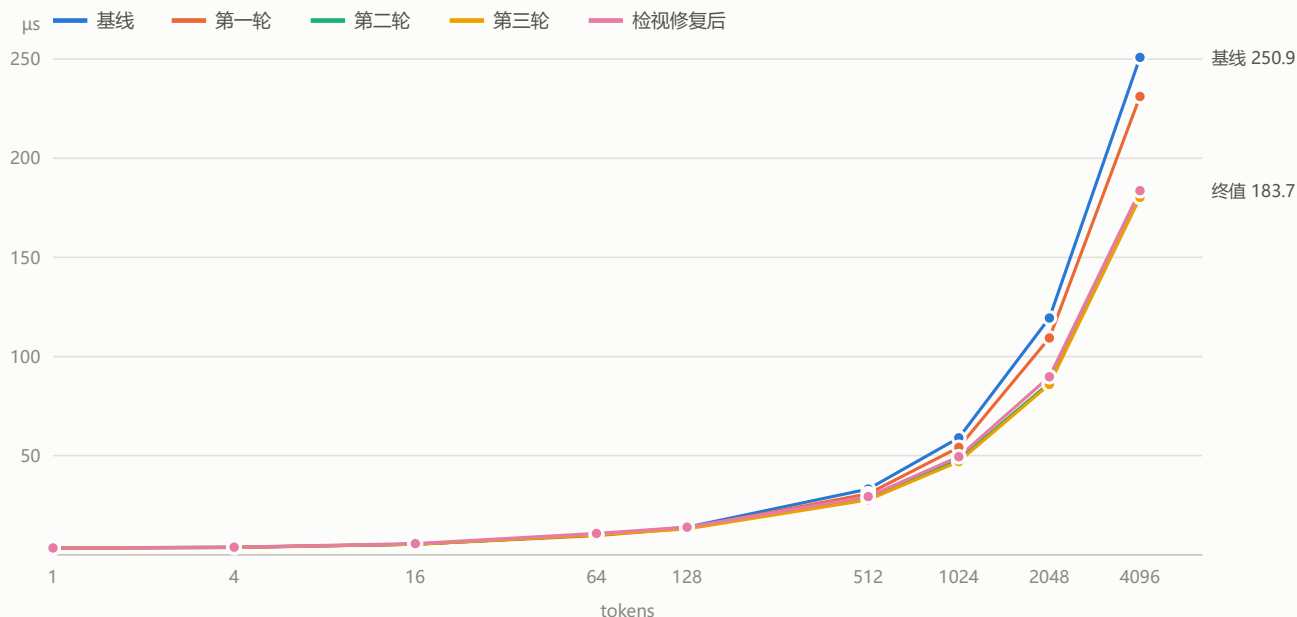


图 5 · 各版本时延演进

fp16 (MULTI_N 折叠保留) : @2048 63.0→60.4 μs (-4.1%) 。 vs unfused ref: bf16 @2048 快 29% (基线仅快 6.5%) 。

最终状态 (msprof, 2048×7168 bf16, 40 核均值, 墙钟 89.4 μs) : vec busy 77.4 μs , `aiv_vec_ratio` 的核间均值为 89.3% (各核 AIV 活跃时间口径) , vec busy 均值占任务墙钟约 86.6%; scalar_wait 1.86 μs , 占墙钟约 2.1%。
MTE2/MTE3 与向量计算大部分重叠 (各 pipe 统计时间非互斥占比) 。\$4.3 的三线并行度 2.87 来自另一组 TimelineDetail 采集, 不与本组 PipeUtilization 读数合并。

关键路径分解 (2048×7168 bf16, 十进制单位) :

项	值	推导
主张量搬运量	117.4 MB	读 $x1+x2$ + 写 $x+y = 4 \times 2048 \times 7168 \times 2B = 117.4$ MB; 近似忽略 rstd 及每核 γ/β 装载
理想 HBM 时间下限	73.4 μs	主张量 117.4 MB $\div$ 名义峰值 1.6 TB/s (设备规格值, 非实测; 不含上述小项)
实测向量占用	77.4 μs	msprof vec busy (占墙钟 86.6%) ; 是当前实现的实测占用, 不是理论下限
实际墙钟	89.4 μs	重叠后的关键路径

按主张量字节量估算, 有效带宽 = $117.4 \text{ MB} \div 89.4 \mu s \approx 1.31 \text{ TB/s}$ (十进制) , 为名义峰值的 ~82%; 结合向量占用, 提示当前实现接近带宽/向量流水转折区, 但不能据此认定带宽是唯一瓶颈。墙钟与理想 HBM 时间下限的差占墙钟 **17.9%** ($89.4 - 73.4 = 16.0 \mu s$) ; 墙钟与实测向量占用的差占墙钟 **13.4%** ($89.4 - 77.4 = 12.0 \mu s$) , 后者不是理论优化空间。ramp/尾行排空、装载/落盘非完美重叠及 DataCopyPad 短粒度是待量化的候选来源, 尚不能拆分归因。

R5 只说明: **本轮 reduce 拆链方案未带来可测的向量占用或墙钟改善**, 并未证明硬件依赖延迟为零, 也不能推广为所有 V 指令令削减都无效。后续应按新的 profiling 证据选择优化点; decode 的 γ/β 并行预取是候选方向, 收益尚未验证。

decode 与 prefill 分列（量级与机制不同，不共用一张纵轴）：

档位	机制	基线 → 终值	说明
decode 1/4/16 tok (SINGLE_N)	发射+延迟混合瓶颈	3.36→3.46 / 3.70→3.87 / 5.32→5.69	μs 级噪声带内；剩余候选 = γ/β 装载并行预取（\$4.4 ^③ 的 SINGLE_N 部分）
prefill 512-4096 tok (NORMAL/MULTI_N)	带宽+向量流水	上表	本报告主体

5. 归因速查表与关键经验

5.1 归因速查表

（本项目实际走过的归因路径；"看什么"是 msprof 字段、tiling 日志或代码检视点）

症状 / 问题	看什么	本例读数 → 结论
不确定优化对象是否命中	ASCEND_GLOBAL_LOG_LEVEL=1 采 tiling 日志的 key/ub_factor	简报说 SPLIT_D，实测 key 30=NORMAL → 重定优化对象
scalar_wait 占比异常高	aiv_scalar_wait_time / wall	85% → 发射线程被标量往返+单缓冲串行钉死，先消往返再行间流水
vec busy 不饱和 + scalar_wait 高	双指标并列读（注明分母）	基线墙钟占比约 70.7% + 85.2% → 减 pass 与流水均有必要；两个指标非互斥占比
收益是真还是噪声	未动路径作金丝雀 ，同 shape、同轮对照并独立复测	第一、二轮以未改 fp16 路径估计噪声，收益判定以 ≥512 档为准；128 档 ±1.2μs 属噪声带，不直接比较不同 shape 的波动百分比
数字看着对但不敢信	精度测试与边界覆盖	补充行间对比、极值、尾块及 UB 边界；精度测试通过
大 shape 收益到顶了？	主张量字节 ÷ 墙钟 vs HBM 峰值	117.4MB/89.4μs ≈ 1.31TB/s，约为名义峰值的 82%；与理想 HBM 时间差约占墙钟 18%，不等于已证明可兑现的收益
"合理"的优化上板无效	实测占用与墙钟对照	本轮 reduce 拆链未观测到明显改善；机理解释待验证，负结果留档
测试结果离奇（超时/崩溃）	/proc/loadavg、设备日志	共享宿主 loadavg 200+（192 核）→ CPU 饥饿，结果不可信，先排环境

5.2 关键经验

- 先实证分发，再动手：**多变体算子的第一件事是用 tiling 日志钉死每个生产 shape 命中的变体与参数（key、ub_factor、block_factor）。本例简报的变体归属是错的——按简报优化将获得精确的零收益；
- 本例优先处理发射线程停等：**基线的逐行 V/S 往返与串行队列是重点排查项；scalar_wait 是停等观测指标，但不能仅凭该字段定位到某一种等待。消除标量往返（Gather+stride-0 广播、rstd 全程 V 域）并建立行间流水后，停等显著减少，减 pass 的收益也更容易兑现；
- bf16 的 pass 下限先算清：**c220 无 bf16 元级乘加 → 所有数学在 fp32 域，加宽/舍入/再宽都是语义必需。可削减的只有"重复计算"（每行重做的 cast、reduce 前的全宽 1/N）；把语义 pass 数数清楚，才知道优化空间还剩多少；
- 未动路径是噪声对照：**在同设备、同 shape、同轮设置下，代码未变路径的波动可辅助估计环境噪声；收益判断还应结合目标路径的独立复测，不能用某个小 shape 的 ±10% 波动直接裁定另一档位的收益；
- 精度验证是性能优化的前提：**新增测试覆盖行间对比、极值输入、尾块与 UB 边界，精度测试通过。每次改变数值路径或缓冲布局，应先完成对应回归，再评价性能收益；

6. **UB 预算与 tiling 必须成本模型化**：常量 ubFactor 往往是"按最坏列宽 × 每列字节"的全宽安全设计；一旦按 numCol 收紧，缓冲总量就随 shape 比例增长——host 侧必须按 dtype/ β 的每列成本 + 固定开销核算，超限回退到已验证路径（并保证回退配置本身落在被验证过的参数域内——本例 colTileNum=1 即回退引入的新雷）；
7. **溢出域也是契约**：数学等价变换仍可能改变舍入路径或溢出域；前者需按算子契约验证，后者必须回退或给出有界证明（fp16 的 max raw sum 远离 fp32 上限，故折叠保留）；
8. **实测结果与机理解释分开留档**：本轮 reduce 依赖链重构收益为噪声级，内部 repeat 机制尚未验证；dst==src1 的交错配置发生 fault、改放 src0 后通过，是当前 c220 配置的设备观测，不应外推为所有配置的硬件规律。负结果同样保留，避免重复投入；
9. **环境先于归因**：共享宿主机 loadavg 飙到 200+ 时，测试超时、vector core 异常、离谱波动都可能是 CPU 饥饿的产物——`/proc/loadavg` 一条命令先排除环境，再怀疑 kernel。